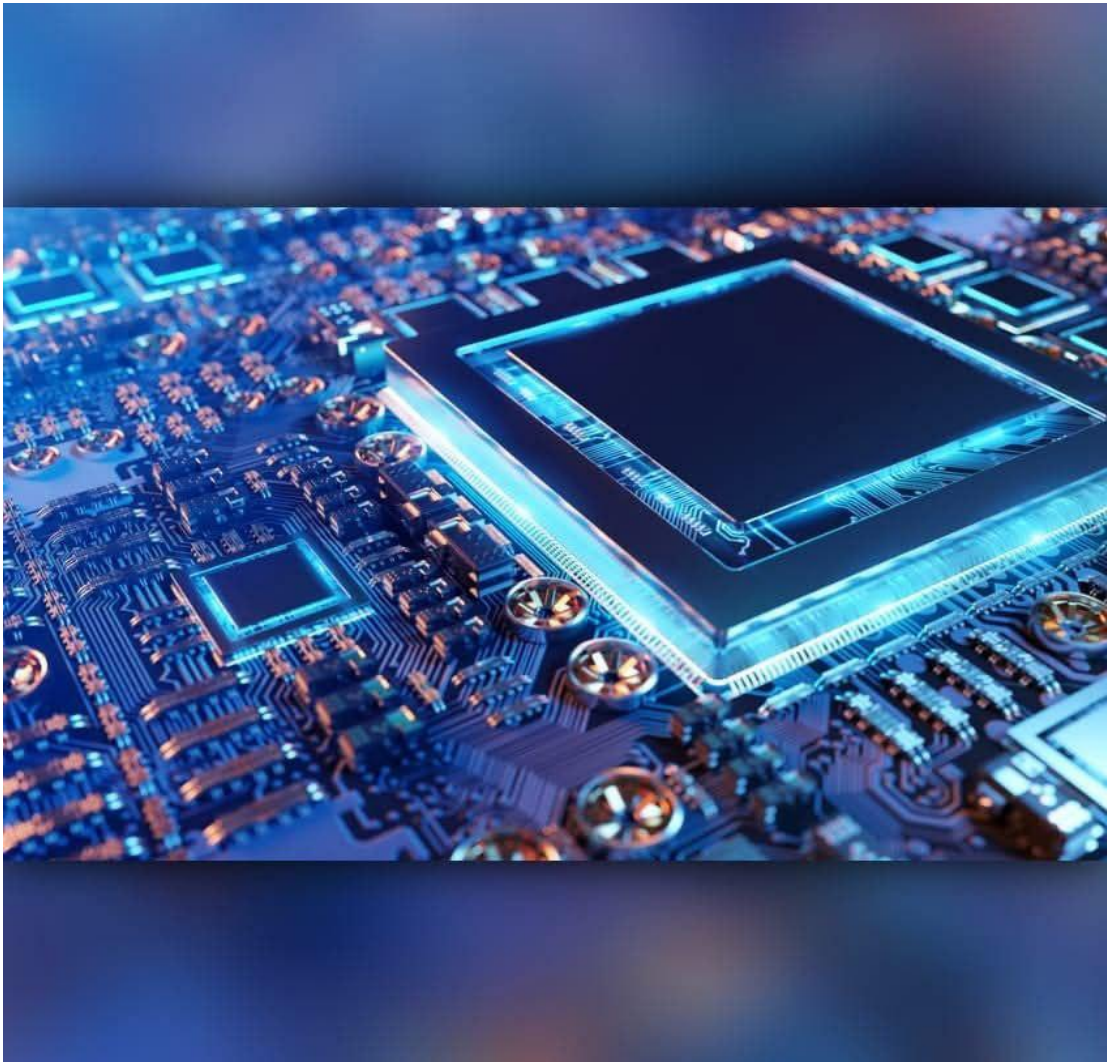


Digital Electronics Design
Assignment_Three_Extra

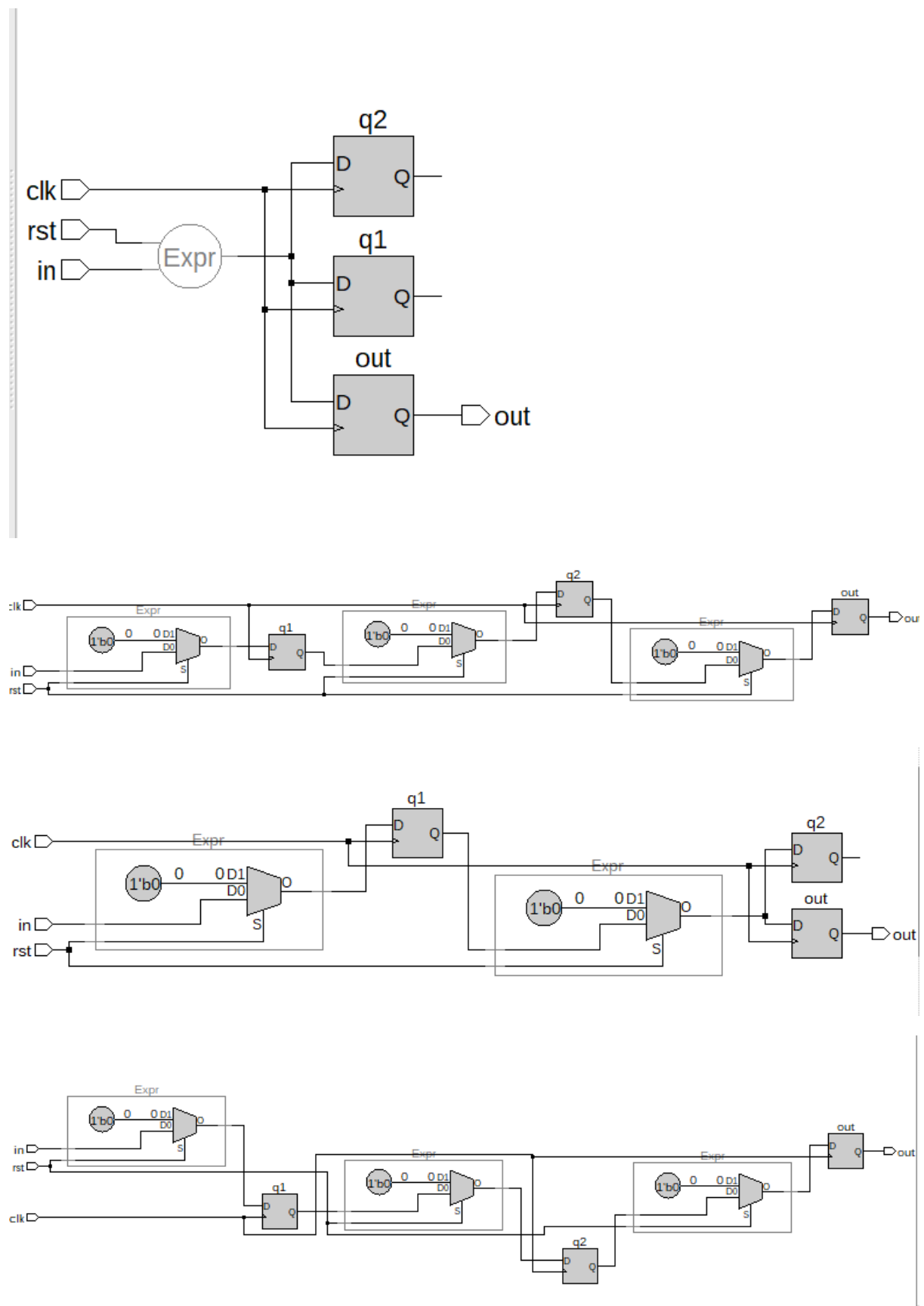
Prepared by:

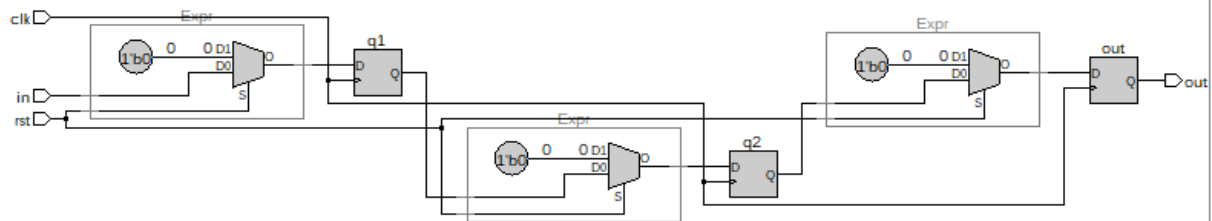
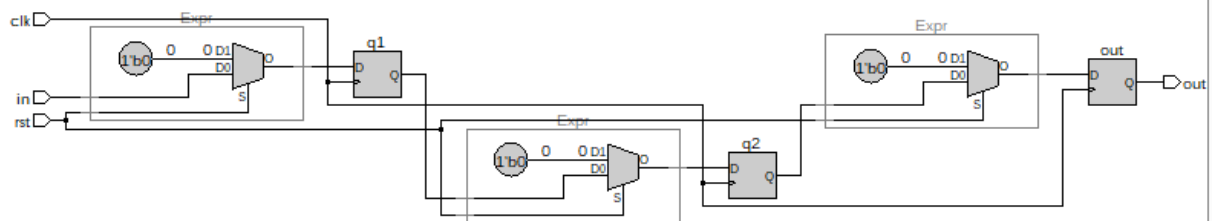
Omar Ahmed Abdelaty Mohammed



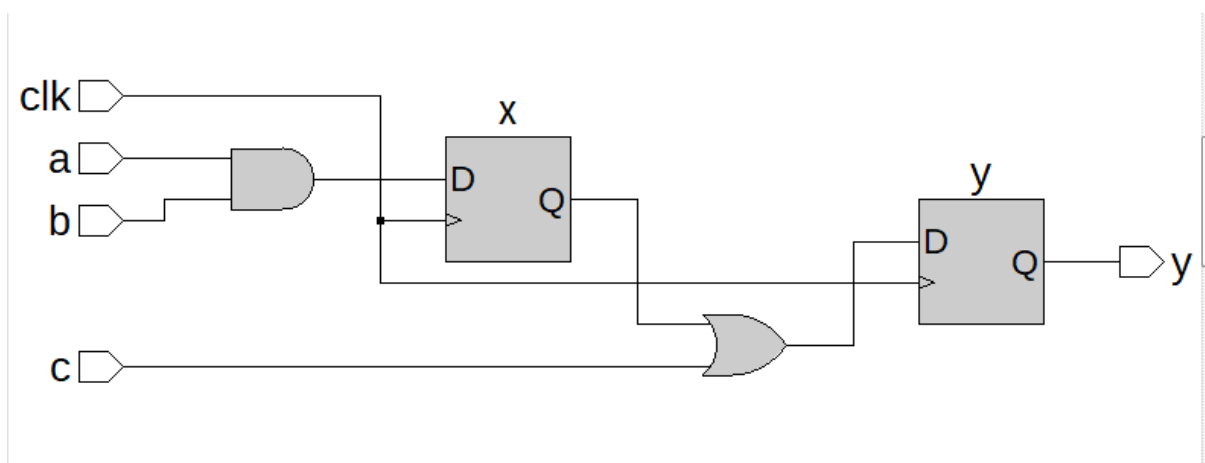
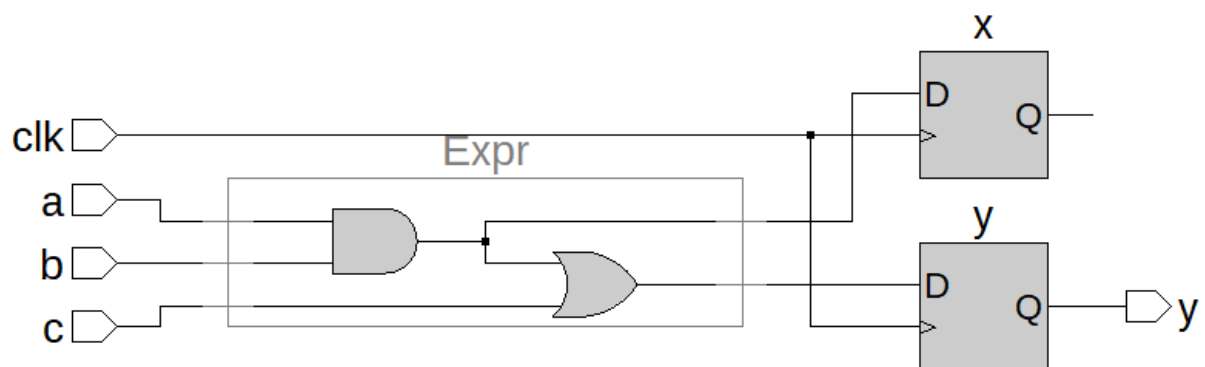
Submitted to: Eng. Kareem Waseem

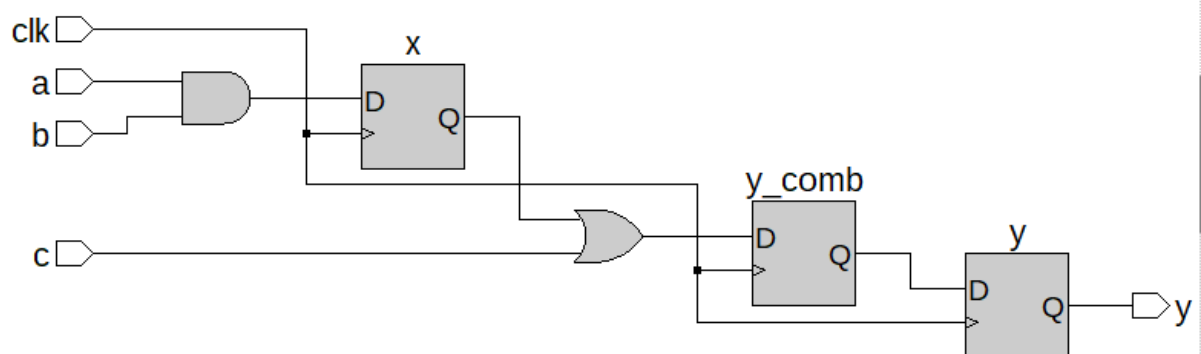
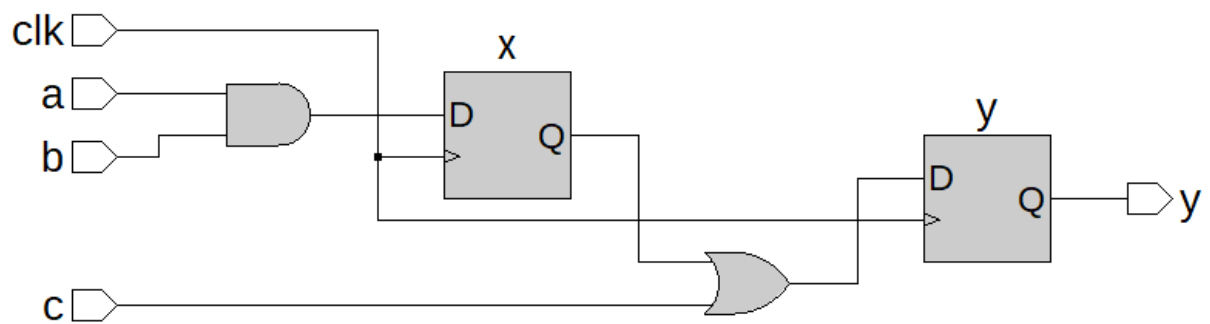
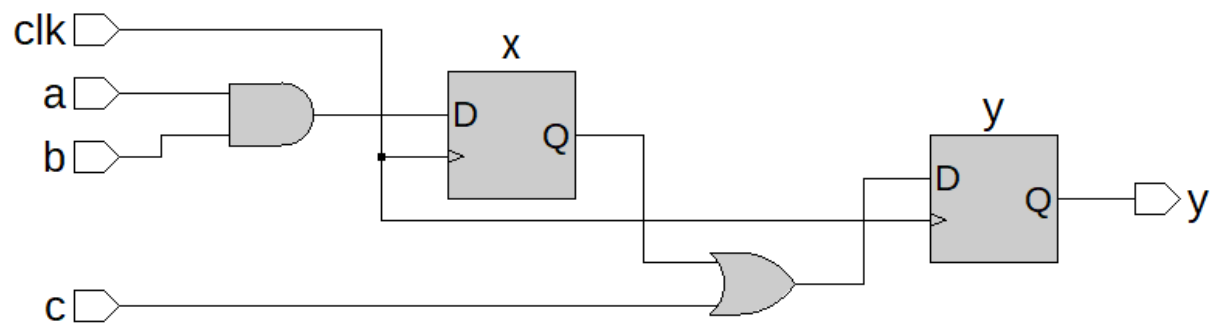
Question 1 from Questalint



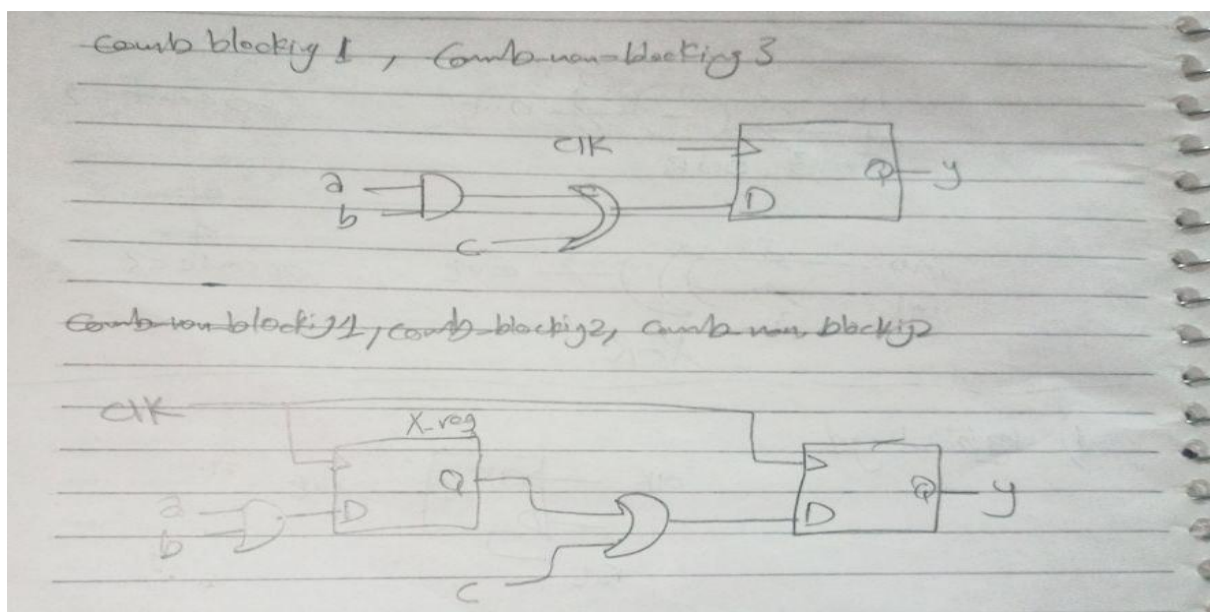
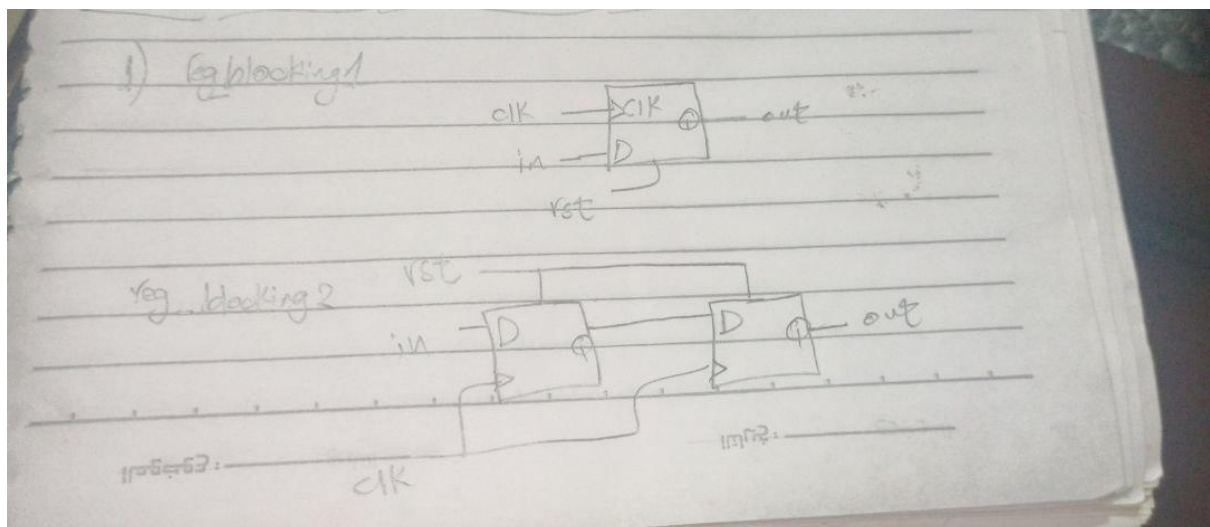
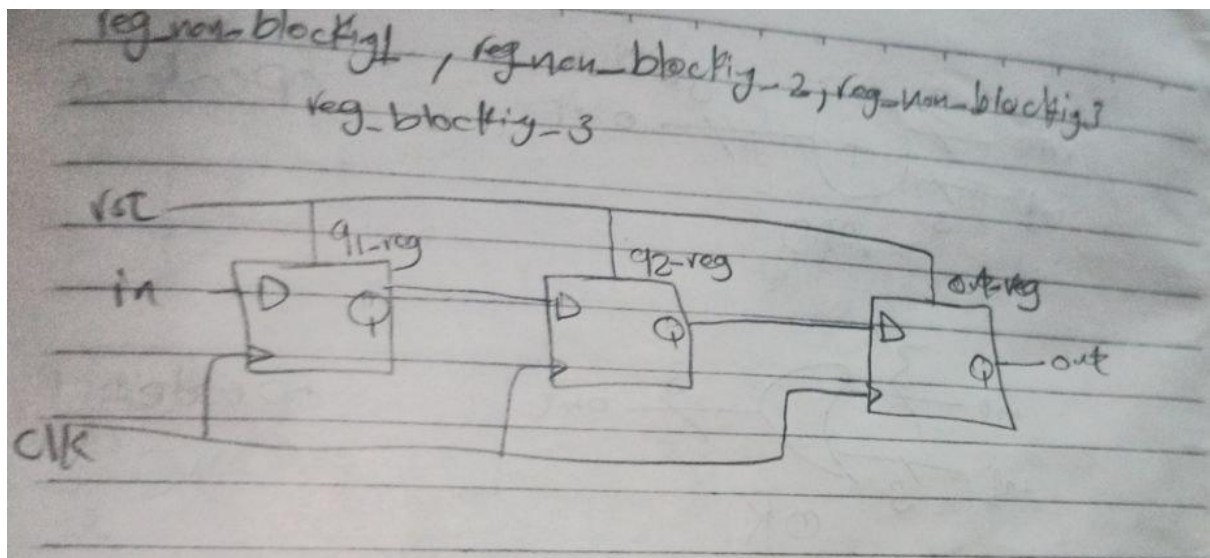


2

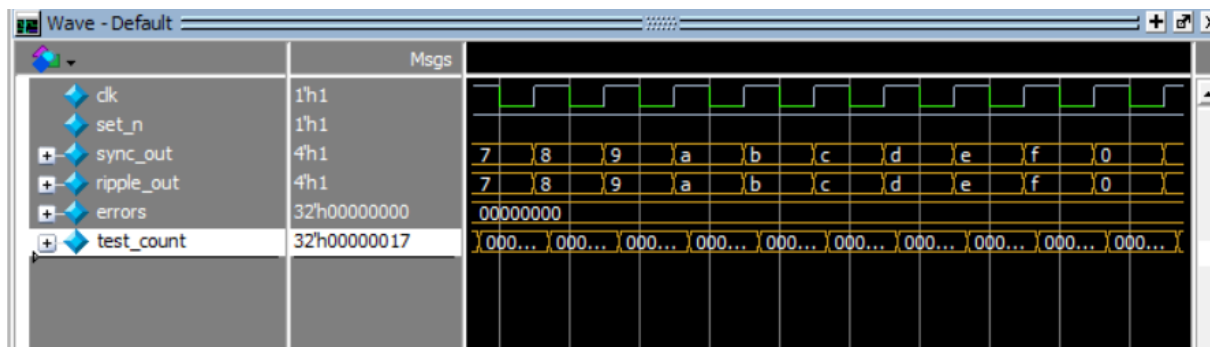




Hand Drawn Schematics



Question 2





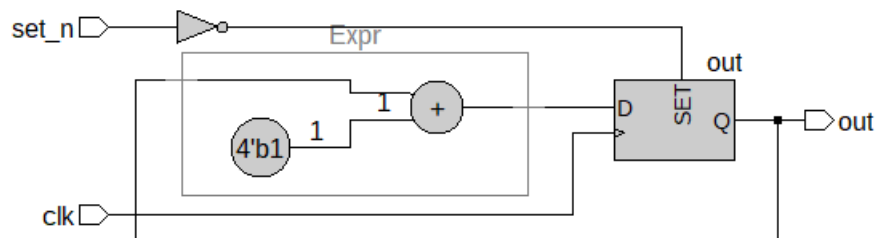
```
1  module sync_counter_4bit (  
2      input clk,  
3      input set_n,  
4      output reg [3:0] out  
5  );  
6  
7  
8      always @(posedge clk or negedge set_n) begin  
9          if (!set_n) begin  
10  
11              out <= 4'b1111;  
12              end else begin  
13  
14                  out <= out + 1;  
15              end  
16          end  
17  
18  endmodule
```



```
1  module dff_rstn (  
2      input clk,  
3      input rstn,  
4      output reg q,  
5      output qn  
6  );  
7  
8      assign qn = ~q;  
9  
10     always @(posedge clk or negedge rstn) begin  
11         if (!rstn) begin  
12             q <= 1'b1;  
13         end else begin  
14             q <= ~q;  
15         end  
16     end  
17  
18 endmodule
```

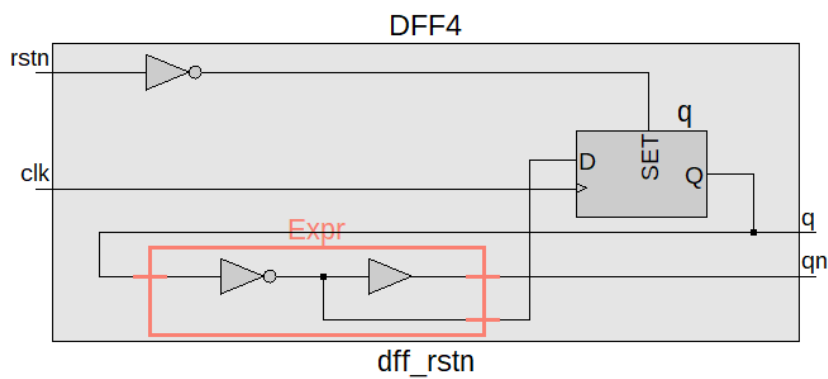


```
1  module ripple_counter (  
2      input clk,  
3      input rstn,  
4      output wire [3:0] out  
5  );  
6  
7      wire gn0, gn1, gn2;  
8  
9      dff_rstn DFF1 (.clk(clk), .rstn(rstn), .q(out[0]), .qn(gn0));  
10     dff_rstn DFF2 (.clk(gn0), .rstn(rstn), .q(out[1]), .qn(gn1));  
11     dff_rstn DFF3 (.clk(gn1), .rstn(rstn), .q(out[2]), .qn(gn2));  
12     dff_rstn DFF4 (.clk(gn2), .rstn(rstn), .q(out[3]), .qn());  
13  
14 endmodule
```



Lint Checks

Filter: Type here Waived Fixed Pending 2 Uninspected Bug Verified Total : 0 Selected : 0

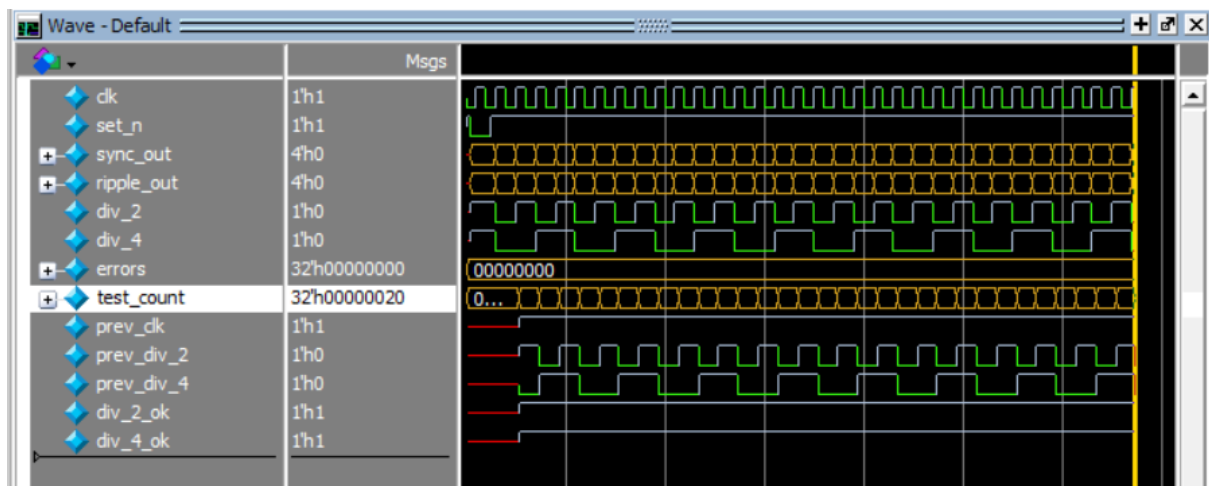



```

64 module testbench;
65     reg clk, set_n;
66     wire [3:0] sync_out, ripple_out;
67     integer errors = 0, test_count = 0;
68
69     sync_counter_4bit sync_counter (.clk(clk), .set_n(set_n), .out(sync_out));
70     ripple_counter ripple_counter (.clk(clk), .rstn(set_n), .out(ripple_out));
71
72     initial begin clk = 0; forever #5 clk = ~clk; end
73
74     initial begin
75         $display("Time\tset_n\tSync_Out\tRipple_Out\tMatch");
76         set_n = 1; #2;
77
78         set_n = 0; #10; check_outputs("Set");
79         set_n = 1; #5;
80         repeat (20) begin #10; check_outputs("Count"); end
81
82         set_n = 0; #5; check_outputs("Async set");
83         set_n = 1; #5;
84         repeat (16) begin #10; check_outputs("Wraparound"); end
85
86         $display("\nTests: %d, Errors: %d", test_count, errors);
87         if (errors == 0) $display("*** ALL TESTS PASSED ***");
88         else $display("*** %d TESTS FAILED ***", errors);
89         $stop;
90     end
91
92     task check_outputs;
93         input [80*8-1:0] test_name;
94         begin
95             test_count = test_count + 1;
96             $display("%0t\t%b\t%4b\t%8b\t%s", $time, set_n, sync_out, ripple_out,
97                 (sync_out == ripple_out) ? "PASS" : "FAIL");
98             if (sync_out != ripple_out) begin
99                 errors = errors + 1;
100                 $display("ERROR in %s: Expected %b, Got %b", test_name, ripple_out, sync_out);
101             end
102         end
103     endtask
104
105     initial begin #2000; $display("Test timeout!"); $stop; end
106 endmodule

```

Question 3



```
1 module sync_counter_4bit_extended (  
2     input clk, set_n,  
3     output reg [3:0] out,  
4     output div_2, div_4  
5 );  
6     always @(posedge clk or negedge set_n) begin  
7         if (!set_n) out <= 4'b1111;  
8         else out <= out + 1;  
9     end  
10  
11     assign div_2 = out[0]; // LSB toggles every clock = clk/2  
12     assign div_4 = out[1]; // Bit 1 toggles every 2 clocks = clk/4  
13 endmodule
```



```
1 module ripple_counter (  
2     input clk, rstn,  
3     output wire [3:0] out  
4 );  
5     wire gn0, gn1, gn2;  
6     dff_rstn DFF1 (.clk(clk), .rstn(rstn), .q(out[0]), .qn(gn0));  
7     dff_rstn DFF2 (.clk(gn0), .rstn(rstn), .q(out[1]), .qn(gn1));  
8     dff_rstn DFF3 (.clk(gn1), .rstn(rstn), .q(out[2]), .qn(gn2));  
9     dff_rstn DFF4 (.clk(gn2), .rstn(rstn), .q(out[3]), .qn());  
10 endmodule
```

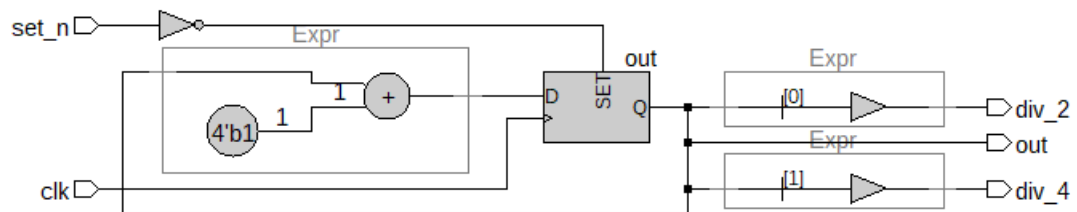


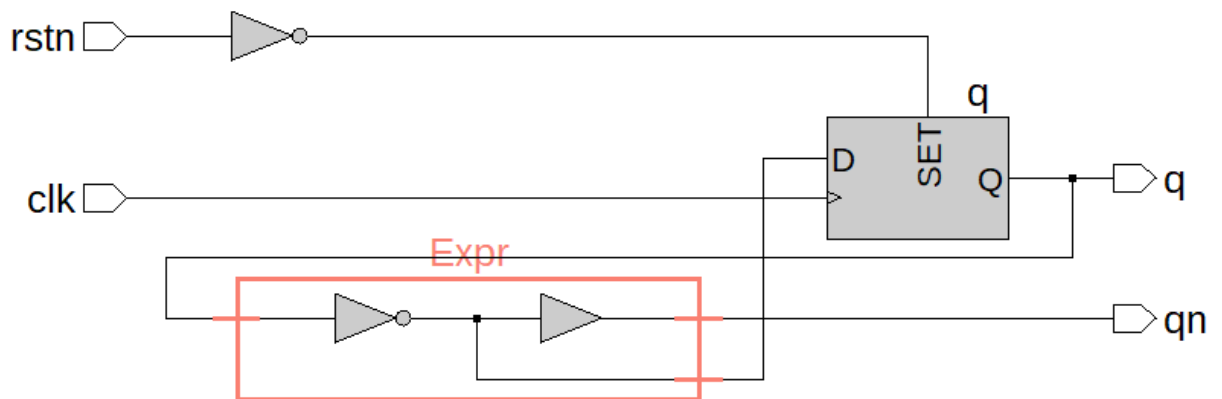
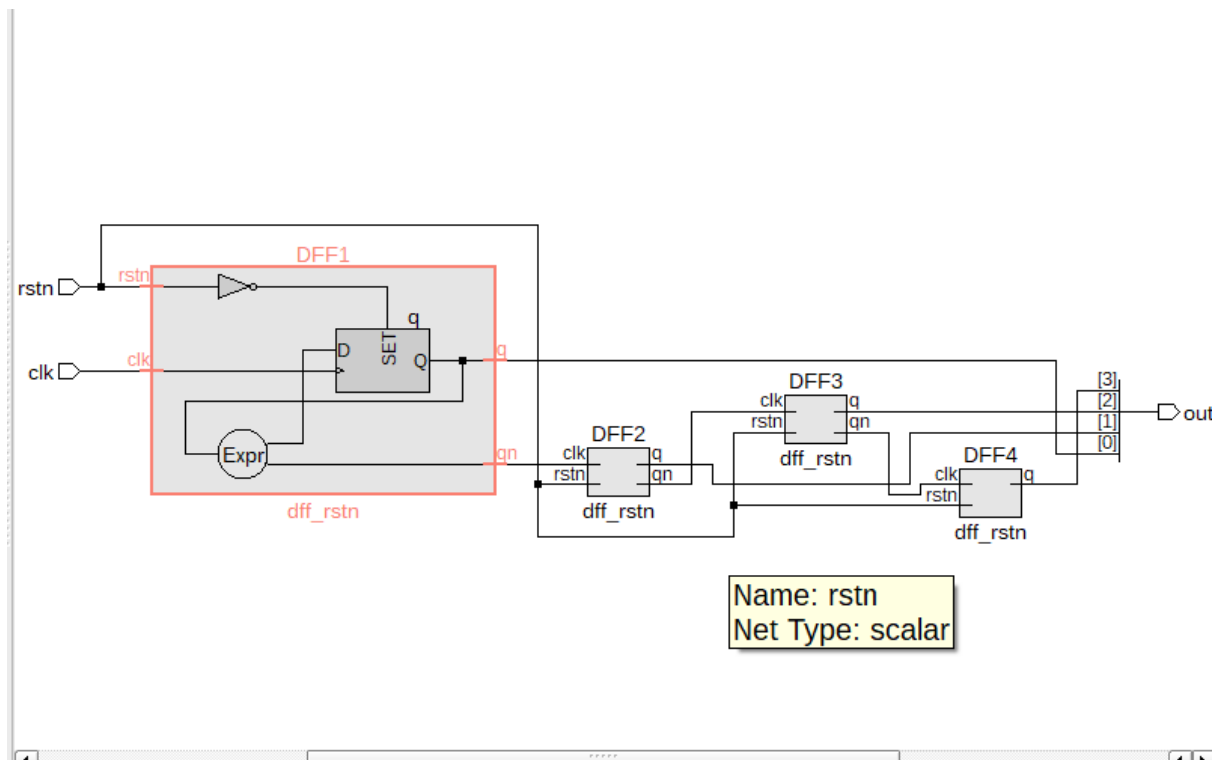
```
1 module dff_rstn (  
2     input clk, rstn,  
3     output reg q,  
4     output qn  
5 );  
6     assign qn = ~q;  
7     always @(posedge clk or negedge rstn) begin  
8         if (!rstn) q <= 1'b1;  
9         else q <= ~q;  
10    end  
11 endmodule
```

```

1  module testbench;
2      reg clk, set_n;
3      wire [3:0] sync_out;
4      wire div_2, div_4;
5
6      sync_counter_4bit_extended counter (.clk(clk), .set_n(set_n), .out(sync_out), .div_2(div_2), .div_4(div_4));
7
8
9      initial begin
10         clk = 0;
11         forever #5 clk = ~clk;
12     end
13
14
15     initial begin
16         $display("Time\tclk\tdiv_2\tdiv_4\tCounter");
17
18
19         set_n = 0; #10;
20         set_n = 1; #5;
21
22
23         repeat (16) begin
24             #10;
25             $display("%0t\tb\tb\tb\tb\tK", $time, clk, div_2, div_4, sync_out);
26         end
27
28         $display("\nTest completed!");
29         $stop;
30     end
31
32
33     initial begin #500; $stop; end
34 endmodule

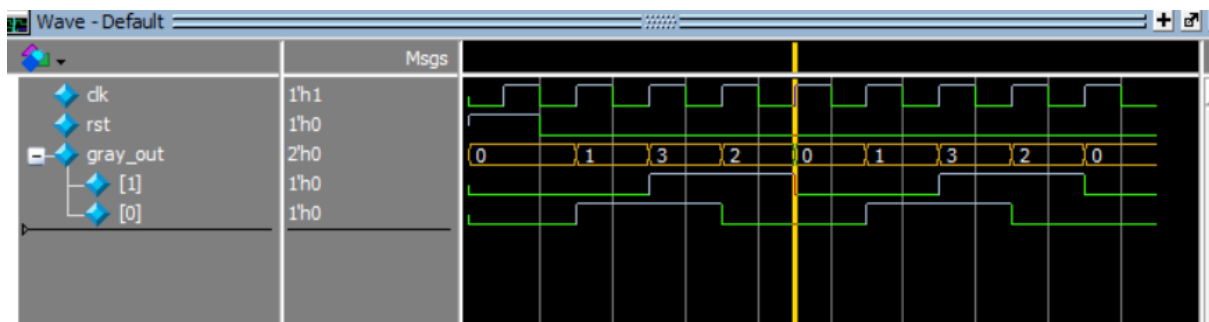
```





Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC R
multi_ports_in_single_line				Multiple ports are declared in one line. Module sync_c... sync_counter...	Rtl Design Style	open	unassign...	3.5.6.3	

Question 4





```
1  module gray_counter (  
2      input clk, rst,  
3      output reg [1:0] gray_out  
4  );  
5      reg [1:0] binary_count;  
6  
7      always @(posedge clk or posedge rst) begin  
8          if (rst) binary_count <= 2'b00;  
9          else binary_count <= binary_count + 1;  
10     end  
11  
12     always @(*) begin  
13         gray_out[1] = binary_count[1];  
14         gray_out[0] = binary_count[1] ^ binary_count[0];  
15     end  
16 endmodule
```

```

1  module testbench5;
2      reg clk, rst;
3      wire [1:0] gray_out;
4
5      gray_counter dut (.clk(clk), .rst(rst), .gray_out(gray_out));
6
7      initial begin
8          clk = 0;
9          forever #5 clk = ~clk;
10     end
11
12     initial begin
13         $display("Time\tclk\trst\tgray_out");
14
15         rst = 1; #10;
16         rst = 0; #5;
17
18         repeat (8) begin
19             #10;
20             $display("%0t\t%b\t%b\t+", $time, clk, rst, gray_out);
21         end
22
23         $stop;
24     end
25 endmodule

```

